LAPORAN PRAKTIKUM
PEMROGRAMAN MOBILE
MODUL 1

ANDROID BASIC WITH COMPOSE

Oleh:

Muhammad Naufal Khalish NIM. 2410817110004

PROGRAM STUDI TEKNOLOGI INFORMASI
FAKULTAS TEKNIK
UNIVERSITAS LAMBUNG MANGKURAT
APRIL 2026

LEMBAR PENGESAHAN

LAPORAN PRAKTIKUM PEMROGRAMAN MOBILE

MODUL 1

Laporan Praktikum Pemrograman Mobile Modul 1: Android Basic With Compose ini disusun sebagai syarat lulus mata kuliah Praktikum Pemrograman Mobile. Laporan Prakitkum ini dikerjakan oleh:

Nama Praktikan : Muhammad Naufal Khalish
NIM : 2410817110004

Menyetujui, Asisten Praktikum	Mengetahui, Dosen Penanggung Jawab Praktikum
Muhammad Erza Raihan NIM. 2310817210027	Irham Maulani Abdul Gani, S.Kom., M.Kom NIP. 199710312025061009

DAFTAR ISI

LEMBAR PENGESAHAN ii

DAFTAR ISI iii

DAFTAR GAMBAR iv

DAFTAR TABEL v

SOAL 1 6

A. Source Code 8

B. Output Program 14

C. Pembahasan 15

ADDITIONAL: ESSAY 17

DAFTAR PUSTAKA 23

TAUTAN GIT 24

DAFTAR GAMBAR

Gambar 1 Tampilan Awal Aplikasi 6
Gambar 2 Tampilan Dadu Setelah Di-Roll 7
Gambar 3 Tampilan Roll Dadu Double 7
Gambar 4 Screenshot Hasil Jawaban Soal 1 XML 14
Gambar 5 Screenshot Hasil Jawaban Soal 1 Compose 14

DAFTAR TABEL

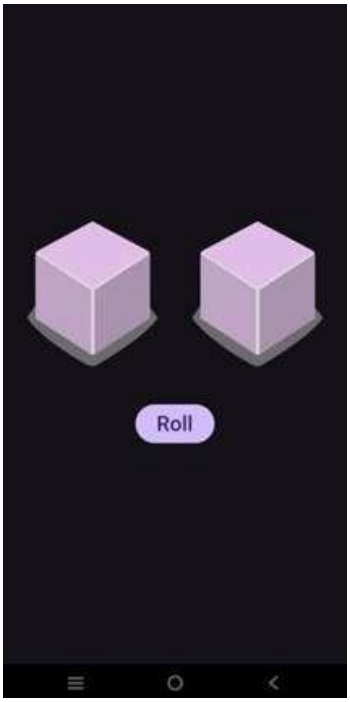
Tabel 1 Source Code MainActivity XML Soal 1 8
Tabel 2 Source Code activity_main XML Soal 1 9
Tabel 3 Source Code MainActivity Compose Soal 1 11

SOAL 1

Soal Praktikum:

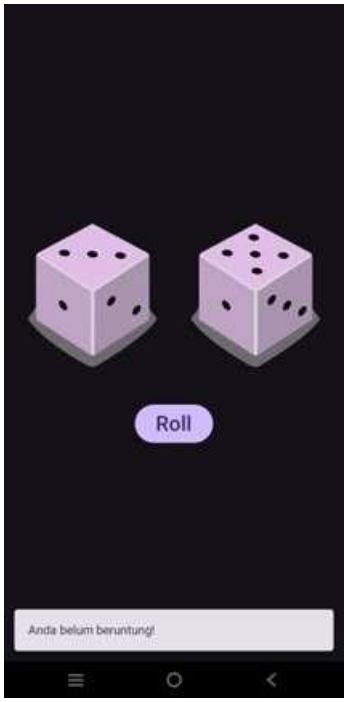
Buatlah sebuah aplikasi yang dapat menampilkan 2 (dua) buah dadu yang dapat berubah ubah tampilannya pada saat user menekan tombol "Roll". Aturan aplikasi yang akan dibangun adalah sebagaimana berikut:

1. Tampilan awal aplikasi setelah dijalankan akan menampilkan 2 buah dadu kosong seperti dapat dilihat pada Gambar 1.



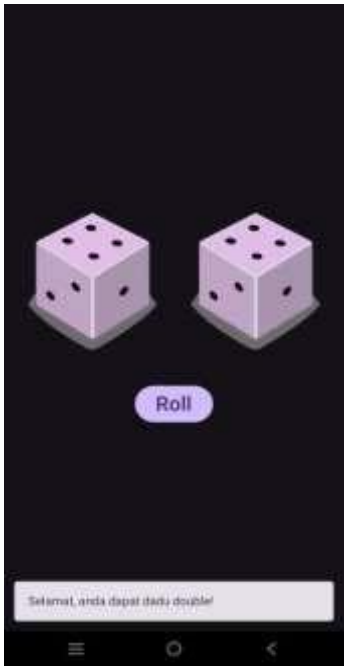
Gambar 1 Tampilan Awal Aplikasi

2. Setelah user menekan tombol "Roll" maka masing-masing dadu akan memperlihatkan sisi dadu masing-masing dengan angka antara 1 s/d 6. Apabila user mendapatkan nilai dadu yang berbeda antara Dadu 1 dengan Dadu 2 maka aplikasi akan menampilkan pesan "Anda belum beruntung!" seperti dapat dilihat pada Gambar 2.



Gambar 2 Tampilan Dadu Setelah Di-Roll

3. Apabila user mendapatkan nilai dadu yang sama antara Dadu 1 dan Dadu 2 atau nilai double, maka aplikasi akan menampilkan pesan "Selamat, anda dapat dadu double!" seperti yang dapat dilihat pada Gambar 3.



Gambar 3 Tampilan Roll Dadu Double

4. Buatlah aplikasi tersbeut menggunakan XML dan Jetpack Compose
5. Upload aplikasi yang telah anda buat ke dalam repository GitHub ke dalam **folder Modul 1 dalam bentuk Project.** Jangan lupa untuk melakukan **Clean Project** sebelum mengupload pekerjaan anda pada repository.
6. Untuk gambar dadu dapat didownload pada link berikut: https://drive.google.com/file/d/14V3qXGdFnuoYN4AGd_9SgFh8kw8X9ySm/view?usp=sharing

A. Source Code

MainActivity.kt - XML

Tabel 1 Source Code MainActivity XML Soal 1

```
package com.example.modul01_xml

import android.os.Bundle
import android.widget.Button
import android.widget.ImageView
import android.widget.Toast
import androidx.appcompat.app.AppCompatActivity

class MainActivity : AppCompatActivity() {

    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        setContentView(R.layout.activity_main)

        val rollbutton : Button = findViewById(R.id.rollButton)
        val diceImage1 : ImageView = findViewById(R.id.diceImage)
        val diceImage2 : ImageView = findViewById(R.id.diceImage2)

        rollbutton.setOnClickListener {
            val number1 = (1..6).random()
            val number2 = (1..6).random()

diceImage1.setImageResource(getImageDice(number1))
diceImage2.setImageResource(getImageDice(number2))

            if (number1 == number2){
                Toast.makeText(this, "Selamat, anda dapat
```

```
dadu double!", Toast.LENGTH_SHORT).show()
            } else {
                Toast.makeText(this, "Anda belum beruntung!", Toast.LENGTH_SHORT).show()
            }
        }
    }

    private fun getImageDice(number : Int): Int {
        return when(number){
            1 -> R.drawable.dice_1
            2 -> R.drawable.dice_2
            3 -> R.drawable.dice_3
            4 -> R.drawable.dice_4
            5 -> R.drawable.dice_5
            else -> R.drawable.dice_6

        }
    }
}
```

activity_main.xml

Tabel 2 Source Code activity_main XML Soal 1

```
<?xml version="1.0" encoding="utf-8"?>

<LinearLayout

xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:tools="http://schemas.android.com/tools"
    android:id="@+id/main"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:orientation="vertical"
    android:gravity="center"
    android:padding="16dp"
    tools:context=".MainActivity">

    <LinearLayout
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:orientation="horizontal">
```

```
19
20          <ImageView
21              android:id="@+id/diceImage"
22              android:layout_width="150dp"
23              android:layout_height="150dp"
24              android:layout_marginBottom="24dp"
25              android:src="@drawable/dice_1" />
26
27          <ImageView
28              android:id="@+id/diceImage2"
29              android:layout_width="150dp"
30              android:layout_height="150dp"
31              android:layout_marginBottom="50dp"
32              android:layout_marginLeft="-15dp"
33              android:src="@drawable/dice_1" />
34
35      </LinearLayout>
36
37          <Button
38              android:id="@+id/rollButton"
40              android:layout_width="wrap_content"
41              android:layout_height="wrap_content"
42              android:text="Roll the Dice"
43              />
44
45  </LinearLayout>
```

MainActivity.kt - Compose

Tabel 3 Source Code MainActivity Compose Soal 1

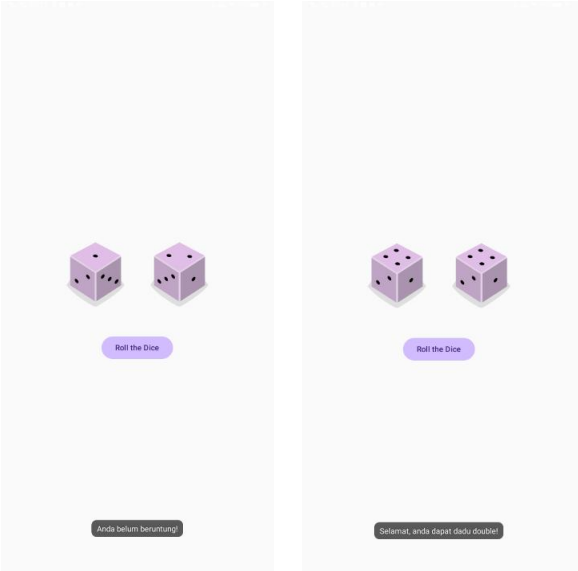
```
package com.example.modul01_compose

import android.os.Bundle
import android.widget.Toast
import androidx.activity.ComponentActivity
import androidx.activity.compose.setContent
import androidx.activity.enableEdgeToEdge
import androidx.compose.foundation.Image
import androidx.compose.foundation.layout.Arrangement
import androidx.compose.foundation.layout.Column
import androidx.compose.foundation.layout.Row
import androidx.compose.foundation.layout.Spacer
import androidx.compose.foundation.layout.fillMaxSize
import androidx.compose.foundation.layout.height
import androidx.compose.foundation.layout.size
import androidx.compose.material3.Button
import androidx.compose.material3.Text
import androidx.compose.runtime.Composable
import androidx.compose.runtime.getValue
import androidx.compose.runtime.mutableIntStateOf
import androidx.compose.runtime.remember
import androidx.compose.runtime.setValue
import androidx.compose.ui.Alignment
import androidx.compose.ui.Modifier
import androidx.compose.ui.platform.LocalContext
import androidx.compose.ui.res.painterResource
import androidx.compose.ui.tooling.preview.Preview
import androidx.compose.ui.unit.dp
import com.example.modul01_compose.ui.theme.Modul01_ComposeTheme
class MainActivity : ComponentActivity() {
    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        enableEdgeToEdge()
        setContent {
            Modul01_ComposeTheme {
                DiceRollerApp()
            }
        }
    }
}

@Composable
```

```
43  fun DiceRollerApp(){
44      val context = LocalContext.current
45      var result by remember { mutableIntStateOf(1) }
46      var result2 by remember { mutableIntStateOf(1) }
47
48      fun getDiceResource(result: Int): Int{
49          return when(result) {
50              1 -> R.drawable.dice_1
51              2 -> R.drawable.dice_2
52              3 -> R.drawable.dice_3
53              4 -> R.drawable.dice_4
54              5 -> R.drawable.dice_5
55              else -> R.drawable.dice_6
56          }
57      }
58
59      Column(
60          modifier = Modifier.fillMaxSize(),
61          horizontalAlignment = Alignment.CenterHorizontally,
62          verticalArrangement = Arrangement.Center
63      ) {
64      Row{
65          Image(
66              painter = painterResource(id =
    getDiceResource(result)),
67              contentDescription = "$result",
68              modifier = Modifier.size(150.dp)
69          )
70          Spacer(modifier = Modifier.height(50.dp))
71
72          Image(
73              painter = painterResource(id =
74  getDiceResource(result2)),
75              contentDescription = "$result2",
                modifier = Modifier.size(150.dp)
76          )
77      }
78
79          Spacer(modifier = Modifier.height(32.dp))
80
81          Button(onClick = {
82              result = (1..6).random()
83              result2 = (1..6).random()
84
85              val text  = if (result == result2) "Selamat,
    anda dapat dadu double!"
86              else "Anda belum beruntung!"
```

```

            Toast.makeText(context, text, Toast.LENGTH_SHORT).show()

                } )
        {
            Text(text = "Roll the Dice")
            }

    }
}

@Preview(showBackground = true)
@Composable
fun DiceRollerPreview() {
    Modul01_ComposeTheme {
        DiceRollerApp()
    }
}
```

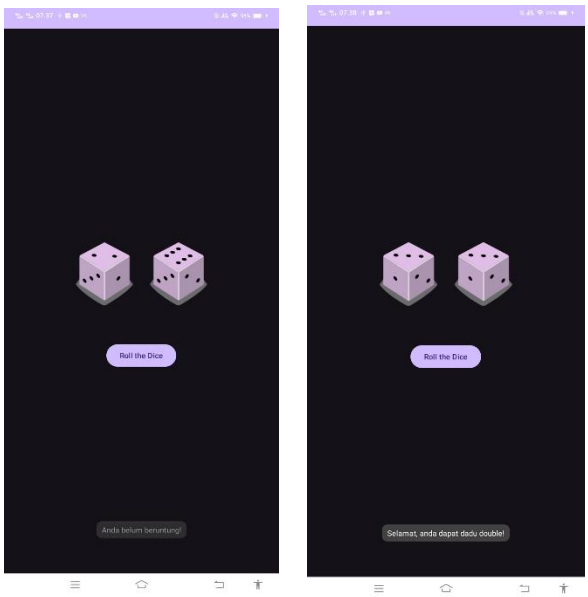
B. Output Program

XML



Gambar 4 Screenshot Hasil Jawaban Soal 1 XML

Compose



Gambar 5 Screenshot Hasil Jawaban Soal 1 Compose

C. Pembahasan

MainActivity.kt - XML

Pada kode MainActivity.kt adalah file kode yang menjalankan logika dari kode aplikasi. Pada kode tersebut untuk melakukan lemparan dua buah dadu yang dimana tiap dadu menghasilkan angka acak dan hasilnya akan dibandingkan dengan dua kondisi yaitu kondisi pada saat masing masing dadu berbeda dan dadu double sama.

Pada bagian `setContentView` menghubungkan file kode ini dengan xml di file yang berbeda bernama `activity_main`. Pada tampilan aplikasi terdiri dari Button sebagai tombol bernama `rollbutton` dan dua ImageView yaitu diceImage1 dan diceImage2 sebagai gambar yang dimana untuk menghubungkan button dan image menggunakan `findViewById`.

Kemudian pada event `setOnClickListener` yaitu pada tombol rollbutton akan menghasilkan angka acak dan diinisialisasi di dua variabel yaitu `number1` dan `number2`, kemudian gambar akan berubah berdasarkan nilai variabel yang dihasilkan acak tadi pada `setImageResource` dengan menggunakan fungsi `getImageDice` yang didalamnya jika nilai yang dihasilkan akan disesuaikan dengan gambar dadu sesuai angkanya. Kemudian terdapat pengkondisian jika double berbeda maka akan menampilkan pop-up pemberitahuan dengan teks "`Selamat, anda dapat dadu double!`" kemudian jika tidak double maka teks nya "`Anda belum beruntung!`"

activity_main.xml

activity_main.xml adalah kode yang mengatur bagaimana tampilan ui aplikasi. Layout utama menggunakan `LinearLayout` yang pertama yang didalamnya menggunakan orientasi vertikal yang hal utama dalam aplikasi yang sering menggunakan orientasi tersebut dan konten diletakkan di posisi tengah layar. Kemudian terdapat `LinearLayout` kedua di dalamnya lagi yang meposisikan orientasi horizontal yang gunanya untuk menempatkan dua gambar dadu yang berdampingan yang ini sesuai dengan dua `ImageView` di dalamnya. Dan terakhir di dalam `LinearLayout` pertama terdapat `Button` yang merupakan tombol yang sebelumnya dihubungkan dengan file kode MainActivity.kt dengan id `rollButton` dengan teks tombol "Roll the Dice"

MainActivity.kt – Compose

Pada file kode ini merupakan compose yang dimana terdapat inti tampilan dan logika aplikasi. Pada penyusunan layout dengan compose ini disusun menggunakan `Column` sebagai layout utama dengan pengaturan posisi berada di tengah layar. Kemudian di dalamnya terdapat `Row` terdapat dua `Image` dadu yang berdampingan. Komponen `Image` digunakan untuk menampilkan gambar berdasarkan nilai yang diinsialisasi yaitu `result` dan `result2` menggunakan `remember` dan `mutableIntStateof` yang gunanya untuk menyimpan nilai hasil lemparan dadu .

Berikutnya ada button yang berisi teks "Roll the Dice" digunakan untuk memulai pelemparan dadu saat diklik dan akan memulai insialisasi nilai dari angka 1 sampai 6 secara acak dan akan disimpan ke masing masing variabel yaitu `result` dan `result2` kemudian jika nilai `result` dan `result2` sama maka akan menampilkan pesan pop up "`Selamat, anda dapat dadu double!`" namun jika berbeda maka teks ditampilkan " Anda belum beruntung!" , pesan pop up ini ditampilkan menggunakn `Toast.`

ADDITIONAL:

ESSAY

1. Riset dan *cite* **artikel/jurnal/buku** mengenai Imperative Programming dan Declarative Programming beserta penerapannya di pembuatan UI. Buatkan opini paradigma apa yang kalian prefer berdasarkan riset yang sudah dilakukan. **
2. Apa yang membedakan OOP pada kotlin dengan OOP pada java? *
3. Sebutkan, dan jelaskan mengenai SOLID Principles *
4. Apa perbedaan antara Class dan Data Class pada Kotlin ?

**** = 2 sumber**
*** = 1 sumber**

Gunakanlah sumber primary artikel/jurnal/buku, web hanya sebagai penunjang opini/argumen

---

1. Imperative dan Declarative Programming

Pengertian

Dalam pemrograman pemrograman imperatif dan deklaratif memiliki practice yang berbeda, Bahasa pemodelan proses imperatif menekankan pada perubahan keadaan secara berkelanjutan dalam suatu proses serta urutan langkah-langkah yang terjadi dari satu kondisi ke kondisi lainnya. Sebaliknya, bahasa pemodelan proses deklaratif berfokus pada logika yang mengatur hubungan antar tindakan dan objek dalam suatu proses, tanpa menjelaskan secara rinci bagaimana proses tersebut dijalankan.[1]. Imperatif lebih memfokuskan bagaimana langkah prosedural dalam mencapai apa yang ingin dihasilkan imperatif dapat ditemui pada bahasa yang sering menggunakan karakteristik perulangan seperti C, C++, Java dan Python sedangkan deklaratif fokus pada hasil akhir ini seringkali digunakan sebagai abstraksi untuk menghindarai perubahan (immutable) hal ini dapat dilihat seperti SQL , HTML , CSS, dan Haskell.

Penerapan pada Pembuatan UI

Graphical User Interaface terdiri dari berbagai elemen yang berinteraksi dan terdapat pada pengembangan software. Meskipun begitu banyak terdapat kesamaan elemen GUI baik mobile ataupun desktop, dalam suatu antarmuka beberapa elemen dapat digantikan dengan fungsi serupa

namun dengan hasil efektif yang berbeda namun beberapa elemen GUI memiliki performa yang berbeda beda di setiap elemen yang digunakan.[2] Oleh karena itu perbandingan performa ini menjadi pertimbangan dalam memilih elemen GUI yang akan digunakan tanpa mengabaikan *usabliity* dan meningkatkan performa. Penerapan kedua paradigma imperatif dan deklaratif memiliki pendekatan berbeda dalam pengembangan GUI. Pendekatan imperatif memberikan kontrol penuh terhadap perubahan elemen secara detail, namun hal ini lebih kompleks. Sdangkan deklaratif bagaimana penyederhanaan atau abstraksi yang memungkinkan pengembang fokus pada hasil akhri tampilan tanpa mengelola detail detail perubahan secare ekspilist.

Kelebihan Dan Kekurangan Paradigma

Paradigma imperatif memiliki keunggulan dalam hal performa. Implementasi imperatif menggunakan lebih sedikit memori dan waktu prosesor dibandingkan OOP. Selain itu kecepatan eksekusi pemrograman prosedural lebih tinggi dari OOP sekitar 8%. Kemudian Pemrograman Deklaratif memiliki keunggulan yang signifikan. Pertama dari sisi semantik, didasarkan pada teori model yang jelas dan formal. Kemudian , pemrograman deklaratif memungkinkan pemrogram hanya menyatakan apa yang ingin dicapai tanpa memikirkan bagaimana cara mencapainya secara teknis. Sehingga pemrogram hanya menuliskan logika atau teori dari masalah yang ingin diselesaikan. Namun terdapat kekurangan dari paradigma ini dari tidak semua aplikasi dimodelkan secara langsung menggunakan pendekatan deklaratif. Meskipun tidak memerlukan informasi kontrol dari pemrogram, kenyataannya pemrogram masig sering menyediakan sebagian informasi kontrol.

Paradigma yang lebih prefer

Jika memilih salah satu paradigma , maka saya memilih paradigma deklaratif karena pemrograman deklaratif selaras dengan cara manusia berpikir secara alami. Yaitu menyatakan apa yang ingin dicapai tanpa harus memikirkan secara rinci bagaimana cara mencapainya. Pendekatan deklaratif semakin relevan dengan perkembangan teknologi modern terutama pengembangan antarmuka. Dalam paradigma ini, program tidak lagi dipandang sebagai sekumpulan instruksi. Hal ini sejalan dengan konsep bahwa program adalah teori dan komputasi merupakan proses deduksi dari teori tersebut.[3]

Selain itu, keunggulan utama dari paradigma deklaratif terletak pada kesederhanaan semantik. Dalam kaitannya dengan pemrogarman deklaratif, konsep multiparadigma sangat

relevan karena dapat hadir sebagai menggabungkan deklaratif sebagai penyederhanaan logika. Banyak framework modern secara tidak langsung menerapkan multiparadigma.Misalnya ,dalam pengembangan UI menggunakan React atau Jetpack Compose, bagian tampilan ditulis secara deklaratif, tetapi logika bisnis, pengelolaan event, dan interaksi sistem seringkali tetap menggunakan pendekatan imperatif

2. Perbedaan Kotlin dan Java dalam OOP

Kotlin dan Java sama-sama merupakan bahasa pemrograman berbasis JVM yang mendukung paradigma pemrograman berorientasi objek (OOP)[4] , namun keduanya memiliki sejumlah perbedaan mendasar dalam implementasinya. Dari sisi deklarasi kelas, Java mengharuskan penulisan field, konstruktor, getter, dan setter secara eksplisit, sedangkan Kotlin menyederhanakan hal ini melalui primary constructor dan *data class* yang secara otomatis menghasilkan metode equals(), hashCode(), toString(), dan copy() tanpa kode tambahan. Dalam hal pewarisan, Java mengizinkan semua kelas diwariskan secara default kecuali dinyatakan final, sementara Kotlin menerapkan kebalikannya di mana semua kelas bersifat final secara default dan hanya dapat diwariskan jika ditandai open, mendorong prinsip *composition over inheritance*. Dari sisi enkapsulasi, Kotlin menggantikan akses package-private Java dengan modifier internal yang membatasi visibilitas dalam satu modul, serta tidak mengenal konsep field secara langsung melainkan menggunakan properti yang lebih terstruktur. Selain itu, Kotlin menghadirkan fitur OOP modern yang tidak dimiliki Java secara bawaan, seperti *sealed class* untuk hierarki kelas tertutup, *companion object* sebagai pengganti metode static, dan null safety yang terintegrasi langsung dalam sistem tipe sehingga NullPointerException dapat dieliminasi sejak tahap kompilasi. Dengan demikian, Kotlin dapat dipandang sebagai evolusi yang lebih modern, ringkas, dan aman dibandingkan Java dalam konteks pemrograman berorientasi objek.

3. SOLID Principles

Prinsip SOLID adalah sebuah pedoman desain OOP yang memiliki lima pedoman yang digunakan untuk membuat kode lebih mudah dipelihara, fleksibel dan terukur. Dari buku *Clean* Architecture : *A Craftsman's Guide to Software Structure and Design* [5] , yang diperkenalkan oleh Robert C. Martin pada tahun 2017.

S : Single Responsibility Principle (SRP)

SRP menyatakan bahwa modul harus memiliki tanggung jawab penuh pada satu. Namun prinsip ini sering disalahartikan sebagai “melakukan satu hal saja”, padahal adalah memisahkan kode agar perubahan bagi satu modul tidak merusak fungsi modul lain yang bergantung pada pihak lain. Pelanggaran SRP sering kali menyebabkan gejala seperti "duplikasi yang tidak disengaja" atau risiko kesalahan saat penggabungan kode (*merge*) karena banyak orang mengubah file yang sama untuk alasan yang berbeda.

O : Open Closed Principle (OCP)

OCP menegaskan bahwa perangkat lunak harus terbuka untuk extension dan tertutup untuk modifikasi. Jika perlikau sistem harus dapat diperluas dengan menambahkan kode baru tanpa harus mengubah kode yang sudah ada dan sudah stabil. Secara arsitektural, hal ini dicapai dengan membagi sistem menjadi komponen-komponen dan mengatur hierarki dependensi sehingga komponen tingkat tinggi.

L : Liskov Substitution Principle (LSP)

LSP menekankan bahwa tipe extend harus dapat disubstitusikan ke dalam tipe dasarnya tanpa merusak perilaku program.pelanggaran klasik pada prinsip ini adalah “Square/Rectangle”, dimana sebuah Square tidak bisa menggantikan Rectangle karena asumsi bahwa mengubah lebar persegi panjang tidak akan mengubah tingginya akan merusak objek persegi.

I : Interface Segregation Principle (ISP)

ISP menyarankan untuk menghindarai ketergantungan pada sesuatu yang tidak digunakan, jika sebuah kelas memiliki banyak operasi yang digunakan oleh aktor yang berbeda, semisal User1 hanya butuh op1, User2 butuh op2, maka perubahan pada op2 akan memaksa User1 untuk dikompilasi ulang meskipun kebutuhan tidak berubah. Masalah ini dapat diselesaikan dengan

memecah interface yang tertumpuk menjadi interface yang lebih spesifik.

D : Dependency Inversion Principle (DIP)

DIP menyatakan bahwa modul tingkat tinggi tidak boleh bergantung pada modul tingkat rendah; keduanya harus bergantung pada abstraksi, dan abstraksi tidak boleh bergantung pada detail, melainkan detail yang harus bergantung pada abstraksi. Dalam desain tradisional, modul kebijakan tingkat tinggi yang berisi aturan bisnis sering kali bergantung pada modul detail tingkat rendah, sehingga perubahan pada detail dapat merusak kebijakan penting dan membuat kebijakan tersebut sulit digunakan kembali.

4. Perbedaan Antara Class dan Data Class Pada Kotlin

Class di Kotlin

Class di Kotlin adalah blueprint atau template untuk membuat objek. Class digunakan untuk mengenkapsulasi data (properties) dan perilaku (functions) agar kode lebih terstruktur dan bisa digunakan ulang. Cara mendeklarasikannya cukup sederhana, yaitu menggunakan keyword class diikuti nama class-nya. Sebuah class bisa memiliki primary constructor yang ditulis langsung di header class, serta secondary constructor yang ditulis di dalam body class. Primary constructor berfungsi untuk menginisialisasi state awal dari sebuah instance.

Kotlin juga mendukung init block, yaitu blok kode yang dijalankan bersamaan dengan primary constructor saat sebuah instance dibuat. Ini berguna untuk menjalankan logika yang lebih kompleks saat inisialisasi, seperti validasi data menggunakan fungsi require. Selain itu, Kotlin mendukung abstract class yang tidak bisa di-instansiasi secara langsung dan dirancang untuk diwarisi oleh class lain. Abstract class bisa memiliki member abstract maupun non-abstract. Ada juga fitur companion object yang memungkinkan kita mengakses fungsi atau properti tanpa perlu membuat instance dari class tersebut.[6]

Data Class di Kotlin

Data class di Kotlin dirancang khusus untuk menyimpan data. Untuk setiap data class, compiler secara otomatis men-generate fungsi-fungsi tambahan seperti equals()/hashCode(), toString(), fungsi componentN(), dan fungsi copy().

Agar bisa dideklarasikan sebagai data class, ada beberapa syarat yang harus dipenuhi: primary constructor harus memiliki minimal satu parameter, semua parameter primary constructor harus ditandai sebagai val atau var, dan data class tidak bisa bersifat abstract, open, sealed, atau inner.

Salah satu fitur paling berguna dari data class adalah fungsi copy(). Fungsi copy() digunakan untuk menyalin sebuah objek sekaligus mengubah sebagian propertinya, sementara properti lain tetap tidak berubah. Perlu diperhatikan bahwa fungsi copy() hanya membuat *shallow copy*, artinya ia tidak menyalin komponen secara rekursif, sehingga referensi ke objek lain tetap dibagi bersama antara original dan hasil copy-nya.

Fitur lain yang menarik adalah destructuring. Fungsi componentN() yang di-generate secara otomatis memungkinkan data class digunakan dalam destructuring declaration, misalnya val (name, age) = user.[7]

DAFTAR PUSTAKA

[1] D. Fahland *et al.*, “Declarative versus Imperative Process Modeling Languages : The Issue of Understandability,” pp. 1–13.

[2] M. S. Samuel, “An Insight into Programming Paradigms and Their Programming Languages,” vol. 1, no. 1, pp. 37–57, 2017.

[3] M. Alpuente, R. Barbuti, and I. Ramos, “1994 Joint Conference on Declarative Programming GULP-PRODE ’94,” in *Declarative Programming: 1994 Joint Conference, GULP-PRODE’94*, Peñíscola, Spain: Universitat Politècnica de València, 1994.

[4] S. Samuel and S. Bocutiu, *Programming Kotlin*. Birmingham: Packt Publishing, 2017.

[5] R. C. Martin, *Clean Architecture: A Craftsman’s Guide to Software Structure and Design*. Boston, MA: Prentice Hall, 2017.

[6] Kotlin Foundation, “Classes.” [Online]. Available: https://kotlinlang.org/docs/classes.html

[7] Kotlin Foundation, “Data classes.” [Online]. Available: https://kotlinlang.org/docs/data-classes.html

TAUTAN GIT

Berikut adalah tautan untuk repositori GitHub yang telah dibuat

https://github.com/yuuukosoo/Praktikum-Pemrograman-Mobile/tree/main/Modul1